

THE UNIVERSITY OF
SYDNEY

OVER-SAVE: Project Proposal

ELEC5619: Object Oriented Application Frameworks

Tuesday 5pm Group 3

Kartik Nair	(510611749)
Antriksh Dhand	(510415022)
Taozhao Chen	(510019015)
Joey Chen	(520076428)
Aryan Adhikari	(510615596)

Contents

1	Introduction	2
2	Requirements analysis	3
2.1	Use case diagram	3
2.2	Functional requirements	4
2.3	Non-functional requirements	16
3	Database design	18
4	Mockups	20
5	Project plan	28
5.1	Project timeline	28
5.2	Team roles and responsibilities	28
	References	31

1 Introduction

Living with financial stress and anxiety is a constant reality for many parts of our society, especially young adults, students (both domestic and international), and those from lower-socioeconomic backgrounds. Since the COVID-19 pandemic, cost-of-living pressures for housing, food, utilities, and transportation have risen sharply, straining budgets and forcing these vulnerable populations to increasingly live paycheck to paycheck. For example, a 2024 survey found that 55% of Australians aged 15–19 are now very concerned about their financial security (Mission Australia, 2023). Another national report showed that 37% of 23-year-olds reported financial stress in 2023, up from 30% the year prior (National Centre for Vocational Education Research (NCVER), 2023). Despite working hard or receiving scholarships, small emergencies, peer pressure, or even the occasional overspending are often enough to derail the financial stability of these groups.

Financial stress is linked to poorer mental health, reduced ability to plan for the future, and fewer opportunities to escape cycles of poverty (Lund et al., 2011). Yet many of those most affected have limited access to budgeting tools that are both reliable and easy to use. Current finance applications are often designed for users who are already financially literate and technically savvy, or are incredibly time-consuming and have complex configurations (Akosah et al., 2023). For young people already overwhelmed by rising costs, these barriers reduce uptake and prevent long-term engagement.

Our group believes this is where there is an opportunity for something novel. We propose to build a budget-tracking application that meets three key goals: simplicity, engagement, and proactivity. Like other finance apps, our system supports the essential features users expect: creating and managing categories, recording expenses and income, tracking recurring subscriptions, setting monthly budgets, defining savings goals, generating reports, exporting/importing data, and viewing rich visualisations on a personal financial dashboard. However, two unique features set our solution apart:

- **Calendar integration:** for example, before an event such as dinner out, users receive a reminder of their budget and suggested spending limits, helping them resist peer pressure or impulse overspending.
- **Gamification:** users earn *BudgetCoins* when they reach savings goals or maintain good financial habits; these coins can be redeemed for rewards, creating positive reinforcement.

Together, these features aim not simply to help people record spending, but to change behaviour in real time. We believe our project addresses a clear gap in the market: most tools are either too technical, too time-consuming, or too disengaging to support those who need help the most.

2 Requirements analysis

2.1 Use case diagram

The use case diagram (Figure 1) illustrates the functional scope and actor interactions for the *OVER-SAVE* personal finance management application. The system enables users to track expenses and income, manage budgets and savings goals, and receive financial insights through a comprehensive web-based platform.

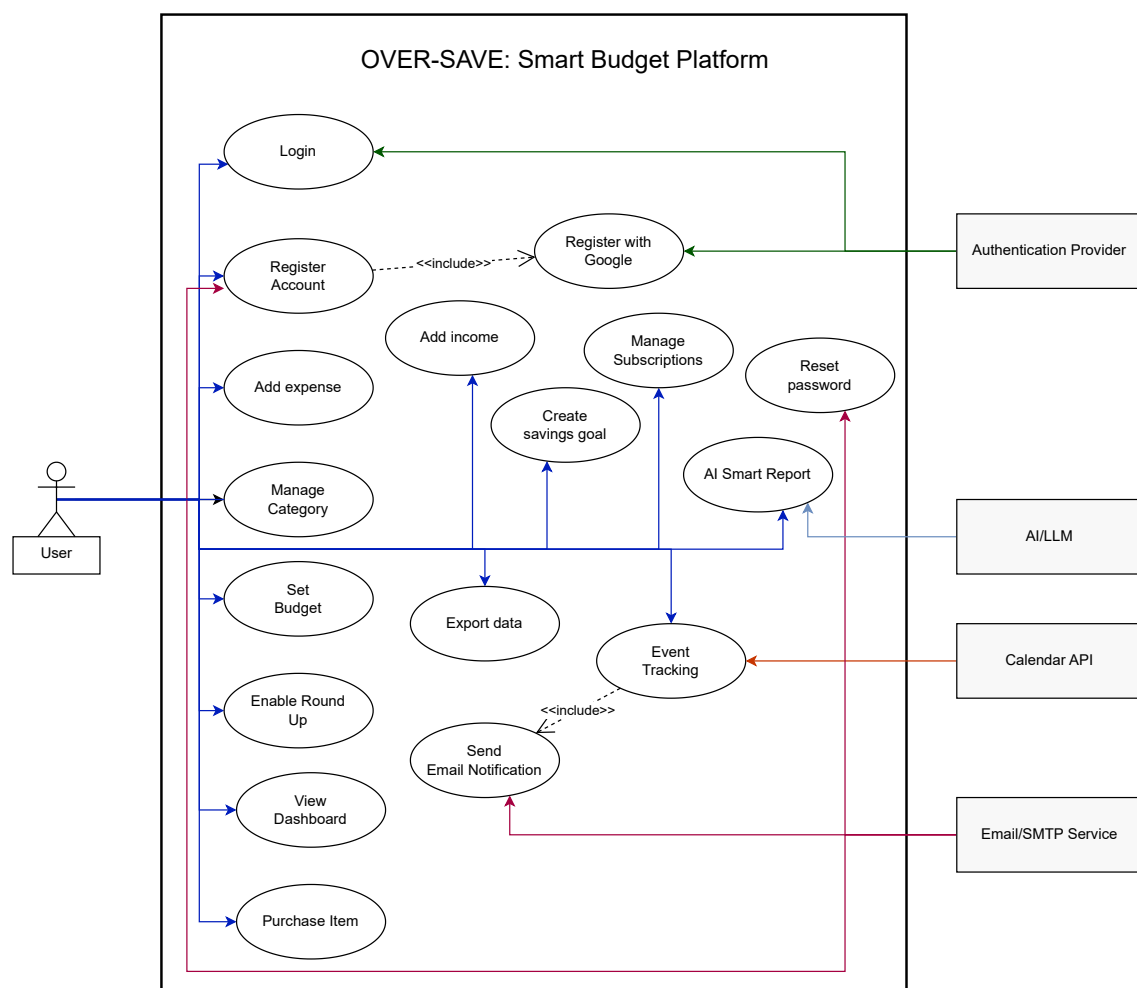


Figure 1: Use case diagram for OVER-SAVE

2.2 Functional requirements

- FR-01: User Registration
- FR-02: Email Verification
- FR-03: Login
- FR-04: Logout
- FR-05: Password Reset
- FR-06: Session Management
- FR-07: Expense Recording
- FR-08: Income Recording
- FR-09: Category Management
- FR-10: Recurring Subscriptions
- FR-11: Budget Setting
- FR-12: Savings Goals
- FR-13: Goal Contributions
- FR-14: Dashboard Display
- FR-15: Savings Progress View
- FR-16: Reports
- FR-17: Data Export
- FR-18: Data Import (Bank Statements)
- FR-19: Email Notification Preferences

FR-1 – User Registration

The system shall allow users to register an account via email/password or supported identity providers (Google, Apple).

Exact sequence of operations

1. User opens app
2. User clicks “Sign Up”
3. User chooses email/password *or* IdP option and completes form
4. System completes email/IdP verification
5. System creates account in the database
6. User is presented with the home screen

Condition

User has not registered previously.

Responses to abnormal situations

- If email already exists, system prompts to log in/reset password.
- If IdP unavailable, fallback to email registration.
- If verification token expired, prompt user to request a new link.

Constraints

OAuth flows must comply with Google/Apple provider policies.

Validity checks on inputs

Email must be unique and valid format. Password must meet strength rules (e.g. at least 12 chars, at least one uppercase letter, at least one number and one special character).

FR-2 – Email Verification

The system shall verify email ownership before activating accounts.

Exact sequence of operations

1. User registers with email
2. System generates a verification token
3. System sends a verification link via email (SMTP)
4. User clicks the link
5. System validates the token
6. System activates the account

Condition

Email registration chosen.

Responses to abnormal situations

- Token expired: system prompts user to request a new link.
- Token invalid: reject and inform user.

Constraints

Verification tokens expire after 24 hours and are single-use.

Validity checks on inputs

Token must match the DB record and must not have been used previously.

FR-3 – Login

The system shall allow users to log in via email/password or IdP.

Exact sequence of operations

1. User navigates to the login page
2. User enters email/password *or* selects an IdP
3. System validates credentials or IdP token
4. System establishes an authenticated session
5. System loads the dashboard

Condition

Account exists (local or IdP-linked).

Responses to abnormal situations

- Invalid credentials: show generic error and allow retry.
- IdP unavailable: suggest alternate method.
- Account locked/disabled: show non-revealing message.

Constraints

Login attempts are rate-limited for a maximum of 5 failed attempts per 15-minute period.

Validity checks on inputs

Email must exist; password hash must match; IdP token must be valid and unexpired.

FR-4 – Logout

The system shall allow users to securely end sessions.

Exact sequence of operations	1. User clicks “Logout” 2. System invalidates the current session token (server-side) 3. Client clears local storage/cache and cookies 4. User is redirected to the login page
Condition	User is authenticated.
Responses to abnormal situations	• Session already expired: client still clears local state and redirects. • Network/server error during logout: client clears local state anyway.
Constraints	Logout only invalidates the current session; other active sessions remain valid.
Validity checks on inputs	Session token must exist before invalidation.

FR-5 – Password Reset

The system shall allow password recovery through secure reset flows.

Exact sequence of operations	1. User clicks “Forgot Password” 2. User enters registered email 3. System generates a secure, time-limited reset token 4. System sends a reset link via SMTP 5. User clicks the reset link and is redirected to the reset form 6. System validates the token 7. User sets a new password
Condition	Account exists and uses local authentication.
Responses to abnormal situations	• OAuth-only account: redirect to IdP recovery. • Token invalid/expired: prompt user to request a new link.
Constraints	Reset tokens expire after 15 minutes; links are single-use.
Validity checks on inputs	Password must be at least 12 characters and include uppercase, numeric, and special characters.

FR-6 – Session Management

The system shall manage session tokens securely.

Exact sequence of operations

1. User logs in
2. System issues a session token
3. Token is renewed on activity, subject to policy
4. Token expires after the configured period
5. User re-authenticates when expired

Condition

User is authenticated.

Responses to abnormal situations

- Expired token: force re-login.
- Tampered token: reject and log the attempt.

Constraints

Idle timeout = 5 minutes

Validity checks on inputs

Tokens must be signed and match the DB record.

FR-7 – Expense Recording

The system shall allow users to add expense transactions.

Exact sequence of operations

1. User clicks “New Transaction → Expense”
2. User fills the form (amount, date, category)
3. System validates inputs
4. System saves the expense
5. Dashboard and reports update

Condition

User authenticated; at least one category exists.

Responses to abnormal situations

- Missing/invalid inputs: show inline error; do not save.
- Network/storage failure: show error and allow retry.

Constraints

Categories must be active/valid.

Validity checks on inputs

Amount > 0; date valid; category exists.

FR-8 – Income Recording

The system shall allow users to add income transactions.

Exact sequence of operations	1. User clicks “New Transaction → Income” 2. User fills the form (amount, date, source) 3. System validates inputs 4. System saves the income record 5. Aggregates and dashboards update
Condition	User authenticated.
Responses to abnormal situations	• Invalid inputs: show error; do not save. • Save operation fails: show error and offer retry.
Constraints	Sources may be tagged but are not required.
Validity checks on inputs	Amount > 0; date valid.

FR-9 – Category Management

The system shall allow creation, editing, deletion, and merging of categories.

Exact sequence of operations	1. User navigates to “Category Management” 2. User creates/edits/deletes/merges categories 3. System validates inputs 4. System saves changes 5. System updates references and dependent views
Condition	User authenticated.
Responses to abnormal situations	• Delete in-use category: prompt user to reassign transactions. • Merge conflict: prompt user to resolve. • DB failure: show error and allow retry.
Constraints	System categories cannot be deleted.
Validity checks on inputs	Unique name; valid characters; length constraints.

FR-10 – Recurring Subscriptions

The system shall allow users to manage recurring subscription expenses.

Exact sequence of operations	1. User adds a subscription (merchant, amount, cadence, start date, category) 2. System validates and saves the rule 3. System auto-generates future entries 4. On due dates, system auto-posts expenses to the ledger
Condition	User authenticated.
Responses to abnormal situations	• Invalid recurrence time-period: reject with validation error. • Auto-post failure: retry later and log error.
Constraints	Only future transactions are generated; past history is retained.
Validity checks on inputs	Amount > 0; cadence valid; start date valid.

FR-11 – Budget Setting

The system shall allow users to set monthly category budgets.

Exact sequence of operations	1. User opens “Budget Settings” 2. User selects a category 3. User enters a monthly limit 4. System validates and saves the budget 5. System recalculates utilisation and updates the dashboard
Condition	User authenticated; category exists.
Responses to abnormal situations	• Invalid amount: show validation error; do not save. • DB failure: show error and allow retry.
Constraints	Budget must be a positive number.
Validity checks on inputs	Amount > 0; numeric; non-empty.

FR-12 – Savings Goals

The system shall allow users to create savings goals.

Exact sequence of operations	1. User selects “Add Goal” 2. User enters name, target amount, optional date 3. System validates inputs 4. System saves the goal 5. System displays progress
-------------------------------------	--

Condition	User authenticated.
------------------	---------------------

Responses to abnormal situations	• Duplicate or missing goal name: reject. • Invalid amount: show error.
---	--

Constraints	User may have multiple goals.
--------------------	-------------------------------

Validity checks on inputs	Amount > 0; unique name required.
----------------------------------	-----------------------------------

FR-13 – Goal Contributions

The system shall allow allocation of income to savings goals.

Exact sequence of operations	1. User selects a savings goal 2. User adds a deposit amount 3. System validates and saves the contribution 4. System recalculates progress
-------------------------------------	--

Condition	User authenticated; goal exists.
------------------	----------------------------------

Responses to abnormal situations	• Invalid allocation: show error. • Database failure: allow retry.
---	---

Constraints	Contribution $\leq$ available balance.
--------------------	--

Validity checks on inputs	Amount > 0; percentage $\leq$ 100%.
----------------------------------	-------------------------------------

FR-14 – Dashboard Display

The system shall provide a dashboard with financial aggregates, budgets, subscriptions, and recent transactions.

Exact sequence of operations

1. User logs in
2. System fetches financial aggregates
3. System loads budgets and recurring subscription data
4. System generates charts and summaries
5. System renders the dashboard

Condition

User authenticated.

Responses to abnormal situations

- Data unavailable: show fallback message.
- Visualisation failure: show textual summary.

Constraints

Dashboard must refresh in real time when new entries are added.

Validity checks on inputs

Data queries must return valid numeric values.

FR-15 – Savings Progress View

The system shall provide visualisation of progress toward savings goals.

Exact sequence of operations

1. User opens “Savings”
2. System fetches goals
3. System calculates progress percentage
4. System renders bars, trend lines, and milestones

Condition

User authenticated; at least one savings goal exists.

Responses to abnormal situations

- No goals: show empty state.
- Data fetch error: retry.

Constraints

Archived goals are displayed separately.

Validity checks on inputs

Goal values must be consistent and numeric.

FR-16 – Reports

The system shall allow generation of financial reports by time period and category.

Exact sequence of operations

1. User selects filters (date range, categories)
2. System retrieves matching transactions
3. System aggregates totals
4. System generates charts and tables
5. System renders the report

Condition

User authenticated; data exists.

Responses to abnormal situations

- No data: show empty state.

Constraints

Reports must generate within 5 seconds.

Validity checks on inputs

Date range valid; category IDs valid.

FR-17 – Data Export

The system shall allow export of data to CSV/Excel. The data will contain a list of all transactions the user has added into the system, as well as summary statistics.

Exact sequence of operations

1. User selects “Export”
2. System validates request
3. System generates CSV/Excel file
4. System provides a download link

Condition

User authenticated; data exists.

Responses to abnormal situations

- Large dataset: system queues background export and notifies user.
- Export failure: show error and allow retry.

Constraints

File size must be ≤ 50 MB.

Validity checks on inputs

Format must be CSV or Excel; date range must be valid.

FR-18 – Data Import (Bank Statements)

The system shall allow users to import bank statements (CSV/Excel).

Exact sequence of operations	1. User selects “Upload Statement” 2. User chooses a file 3. System validates file format 4. System parses the data 5. System stores transactions 6. System shows success summary
Condition	User authenticated; valid file provided.
Responses to abnormal situations	• Invalid file format: reject and prompt re-upload. • Parsing error: abort import and log issue. • Duplicate transactions: skip and inform user.
Constraints	File size ≤ 10 MB; maximum 10,000 rows per upload.
Validity checks on inputs	File must contain required headers: date, description, amount.

FR-19 – Email Notification Preferences

The system shall allow users to manage email notification preferences. Email notifications will include alerts for when the user is close to reaching their budget for a given category or month, and one summary email every month.

Exact sequence of operations	1. User navigates to “Notification Settings” 2. User selects whether to enable or disable email notifications 3. User saves preferences 4. System synchronises with email provider
Condition	User authenticated.
Responses to abnormal situations	• Invalid configuration: reject with error. • Provider unavailable: save locally as pending sync.
Constraints	Preferences apply only to authenticated accounts.
Validity checks on inputs	At least one valid channel must be selected; thresholds must be valid if configured.

FR-20 – BudgetCoin Reward System

The system shall award *BudgetCoins* to users upon achieving predefined financial goals and milestones, enabling redemption for miscellaneous items and services.

Exact sequence of operations

1. System monitors user financial activities
2. System detects goal achievement event and validates achievement legitimacy
3. System calculates reward amount and credits BudgetCoins to user balance
4. System sends the user an achievement notification
5. The user can now put these BudgetCoins towards redeeming items and services in the *OVER-SAVE* store

Condition

User authenticated.

Responses to abnormal situations

- Internal server error while granting coins: queue reward for later processing while notifying the user that something has gone wrong.

Constraints

Coin balance updates must be atomic to prevent inconsistencies.

Validity checks on inputs

Check reward calculation accuracy and check the rate the user is attaining these awards to prevent the manipulation of the gamification system.

2.3 Non-functional requirements

- NFR-01: Performance
- NFR-02: Reliability
- NFR-03: Security
- NFR-04: Usability
- NFR-05: Capacity

NFR-1 – Performance

The system shall respond to user actions within 3 seconds under normal usage.

Acceptance criteria

- Page loads complete within 3 seconds.
- Transaction saves process within 2 seconds.
- PostgreSQL queries execute within 1 second for typical datasets.
- Reports generate within 5 seconds.

NFR-2 – Reliability

The system shall be available 95% of the time during business hours.

Acceptance criteria

- System recovers from failures within 10 minutes.
- PostgreSQL database is backed up daily.
- Application restarts automatically after crashes.

NFR-3 – Security

The system shall protect user financial data from unauthorised access.

Acceptance criteria

- All data encrypted in storage (PostgreSQL) and in transmission (TLS 1.3).
- User sessions expire after 30 minutes of inactivity.
- Passwords stored as salted hashes in PostgreSQL.

NFR-4 – Usability

The system shall be easy to use for non-technical users.

Acceptance criteria

- Works on common web browsers (Chrome, Safari).
- Responsive design for desktop and mobile devices.
- Error messages are clear and actionable.
- Interface loads without requiring plugins or special software.

NFR-5 – Capacity

The system shall support up to 100 concurrent users initially.

Acceptance criteria

- PostgreSQL database handles up to 10,000 transactions per user.
- File uploads limited to 5 MB for bank statement imports.
- Response times increase gracefully as user load approaches capacity limits.

3 Database design

The Entity-Relationship Diagram (ERD) defines the core entities of the *OVER-SAVE* project alongside their attributes and their relationships (Figure 2, [Link to LucidChart](#)).

- **user**: Stores authentication details and user preferences. Includes **budget_coin** for in-app credits. Connected with **idp_account**, **email_verification_token**, **password_reset_token**, and **session**.
- **account**: User-owned accounts (cash or savings/goal accounts) with **balance**. Related to **cash_flow**, **goal**, and **transfer**.
- **cash_flow**: Ledger of income and expenses with **amount**, **occurred_at**, and links to **category**, **account**, and optionally **subscription**.
- **category**: Classifies transactions; may include a **budget** for category-level limits.
- **subscription**: Defines recurring expenses with **merchant**, **amount**, **frequency**, **start_date**, **next_post_date**, and **active**. Can auto-generate entries in **cash_flow**.
- **goal** and **goal_category**: Savings objectives with **target_amount**, optional **target_date**, and **status** (either active or archived). Linked to a specific **account** and organised by categories.
- **transfer**: Records movement of funds (**account_from** to **account_to**) with **amount** and **created_at**.
- **order**: Represents user purchases using BudgetCoins. Includes **address**, **amount**, and references an **item**; associated with a **user**.
- **item**: Catalog of purchasable packages with **item_name**, **price**, and **storage**. Referenced by **order**.

This schema provides secure, scalable, and flexible management of authentication, financial transactions, budgeting, and savings. Constraints such as uniqueness, non-null checks, and default values ensure consistency and integrity.

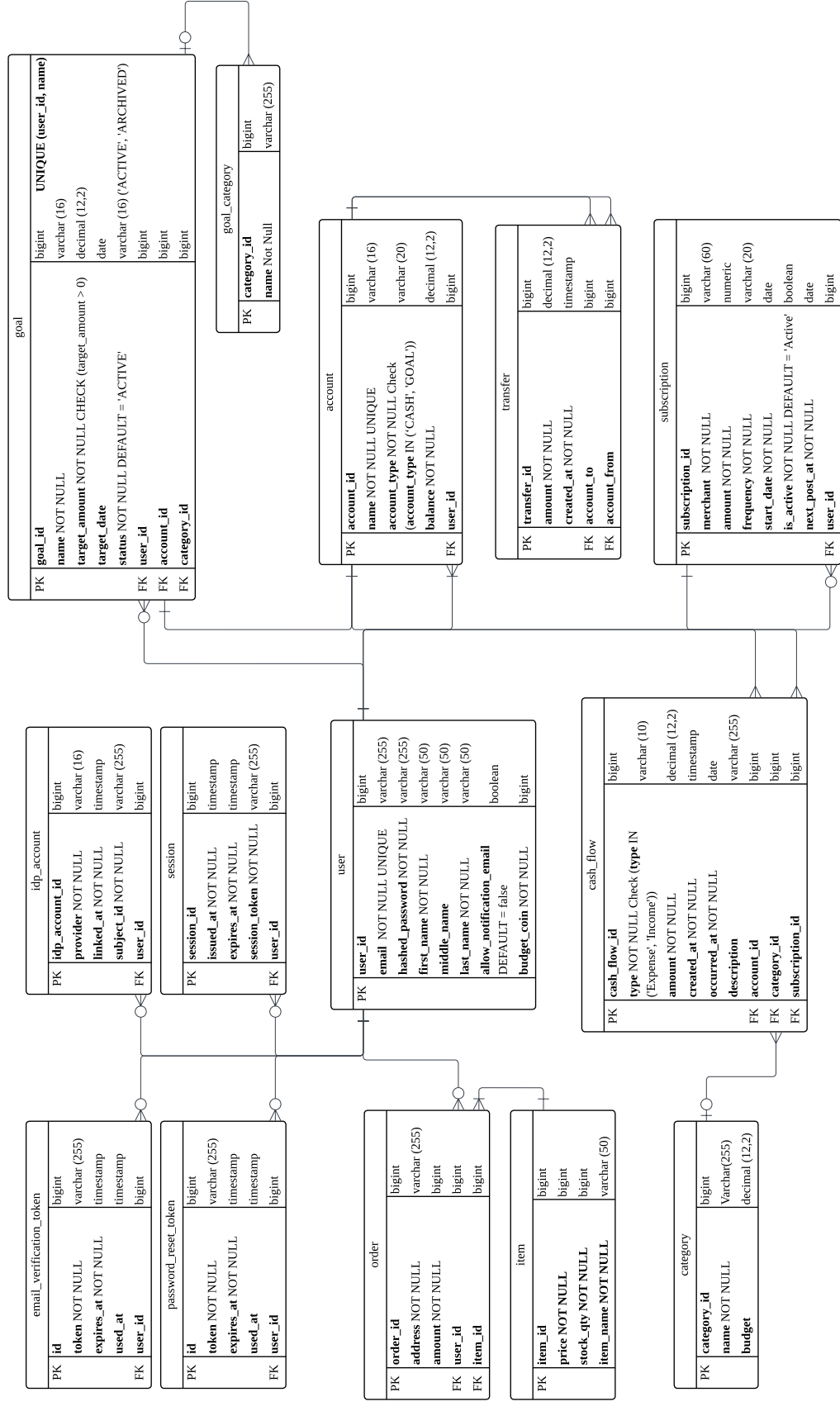


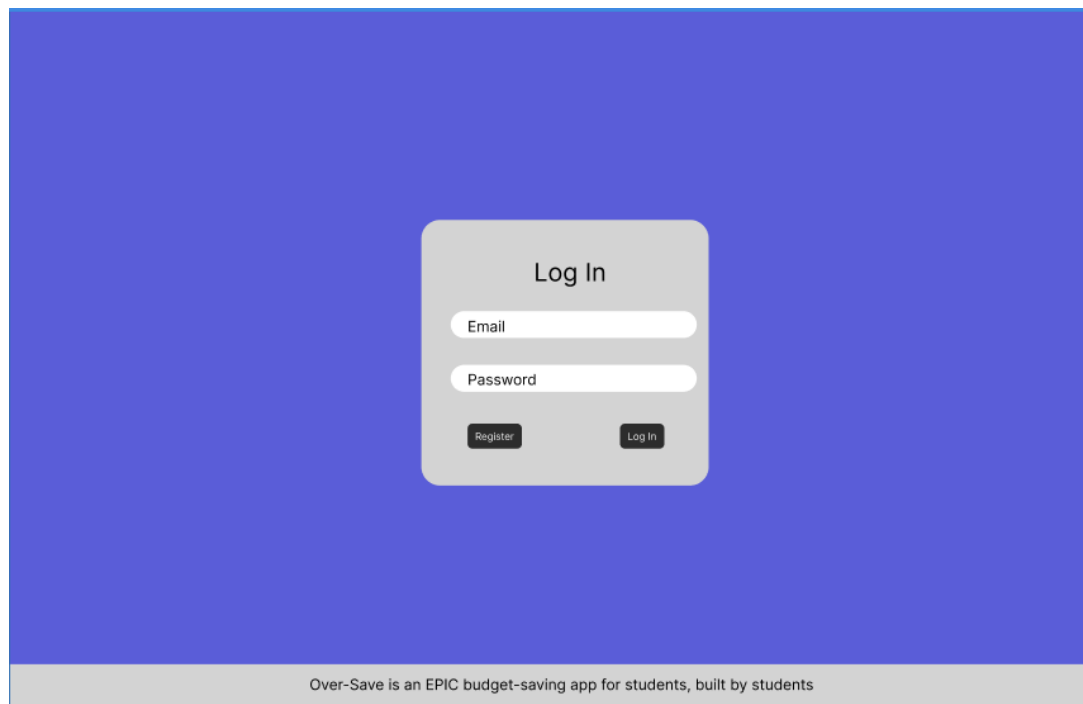
Figure 2: ER diagram for the Over-Save project.

4 Mockups

The following wireframes illustrate the core user journeys and primary interfaces of *OVER-SAVE*. They translate the functional requirements into a simple, low-friction UI that supports quick data entry, clear feedback, and engaging visuals.

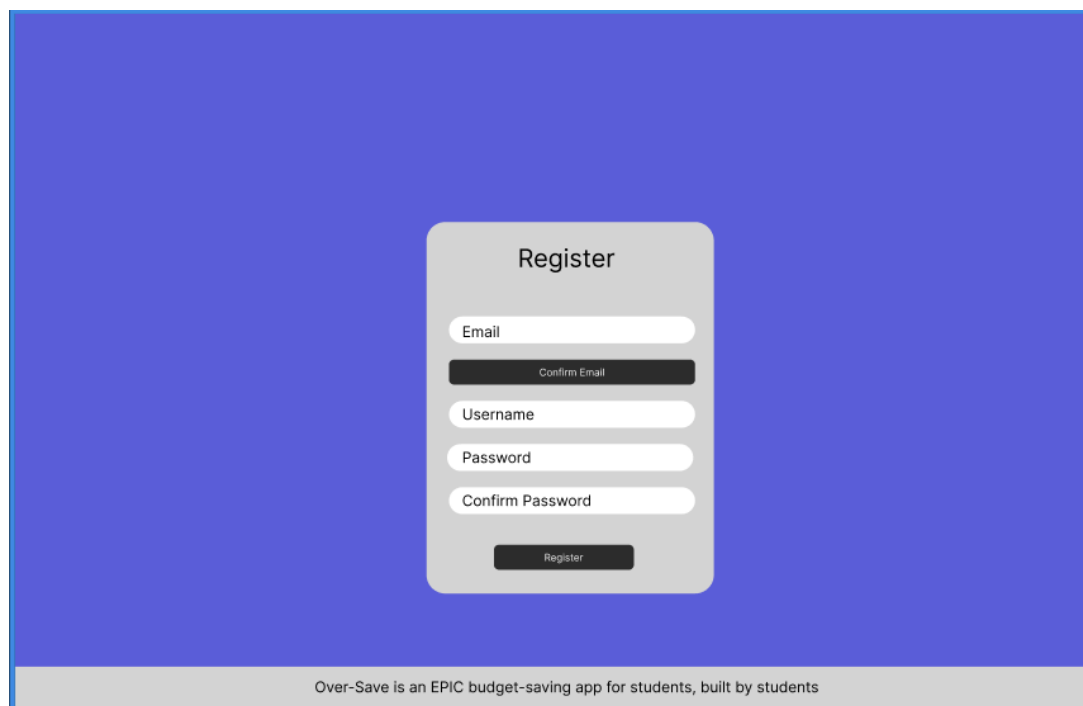
- **Login:** secure email/password authentication (see Figure 3).
- **Register:** quick account creation with essential fields (Figure 4).
- **Home Dashboard:** balances, quick stats, and recent transactions (Figure 5).
- **Analytics:** visualised spending insights (Figure 6).
- **Budget Management:** planned vs. actual by category (Figure 7).
- **Transactions:** unified history for expenses and income (Figure 8).
- **Goals:** create and track savings goals (Figure 9).
- **Rewards Shop:** redeem *BudgetCoins*, reinforcing good habits (Figure 10).

Access to the original Figma board has also been provided [here](#).



The image shows a login screen with a solid blue background. In the center, there is a light gray rounded rectangle containing the text "Log In" at the top. Below it are two white input fields labeled "Email" and "Password". At the bottom of the gray box are two dark gray buttons labeled "Register" and "Log In". A thin gray footer bar at the bottom of the screen contains the text "Over-Save is an EPIC budget-saving app for students, built by students".

Figure 3: Login Screen: Allows users to securely log in to their account with email and password credentials.



The image shows a register screen with a solid blue background. In the center, there is a light gray rounded rectangle containing the text "Register" at the top. Below it are four white input fields labeled "Email", "Confirm Email", "Username", "Password", and "Confirm Password". At the bottom of the gray box is a dark gray button labeled "Register". A thin gray footer bar at the bottom of the screen contains the text "Over-Save is an EPIC budget-saving app for students, built by students".

Figure 4: Register Screen: Enables new users to create an account by providing name, email, and password.

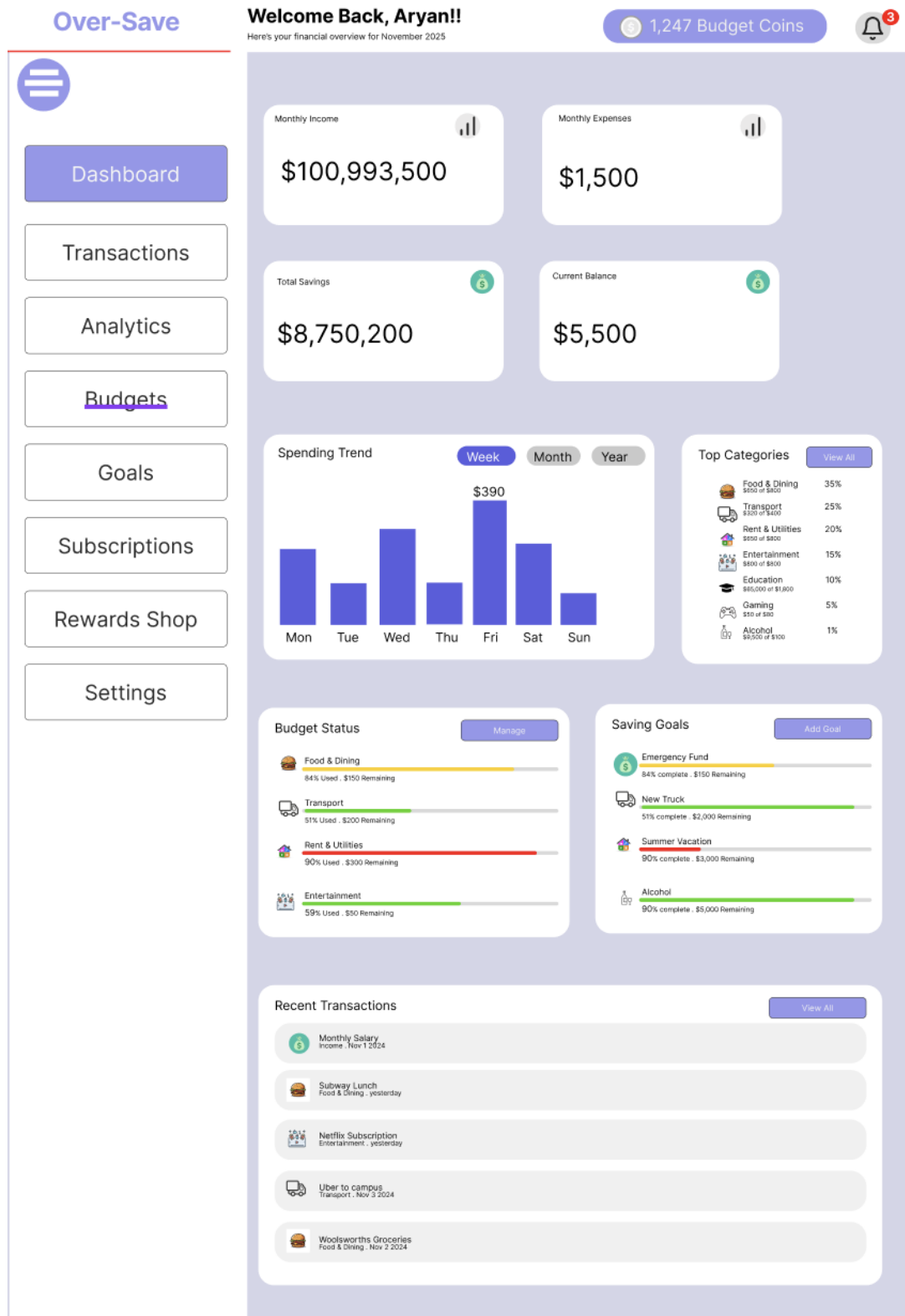


Figure 5: Home Dashboard: Displays account balances, quick statistics, recent transactions, and provides navigation to other sections.

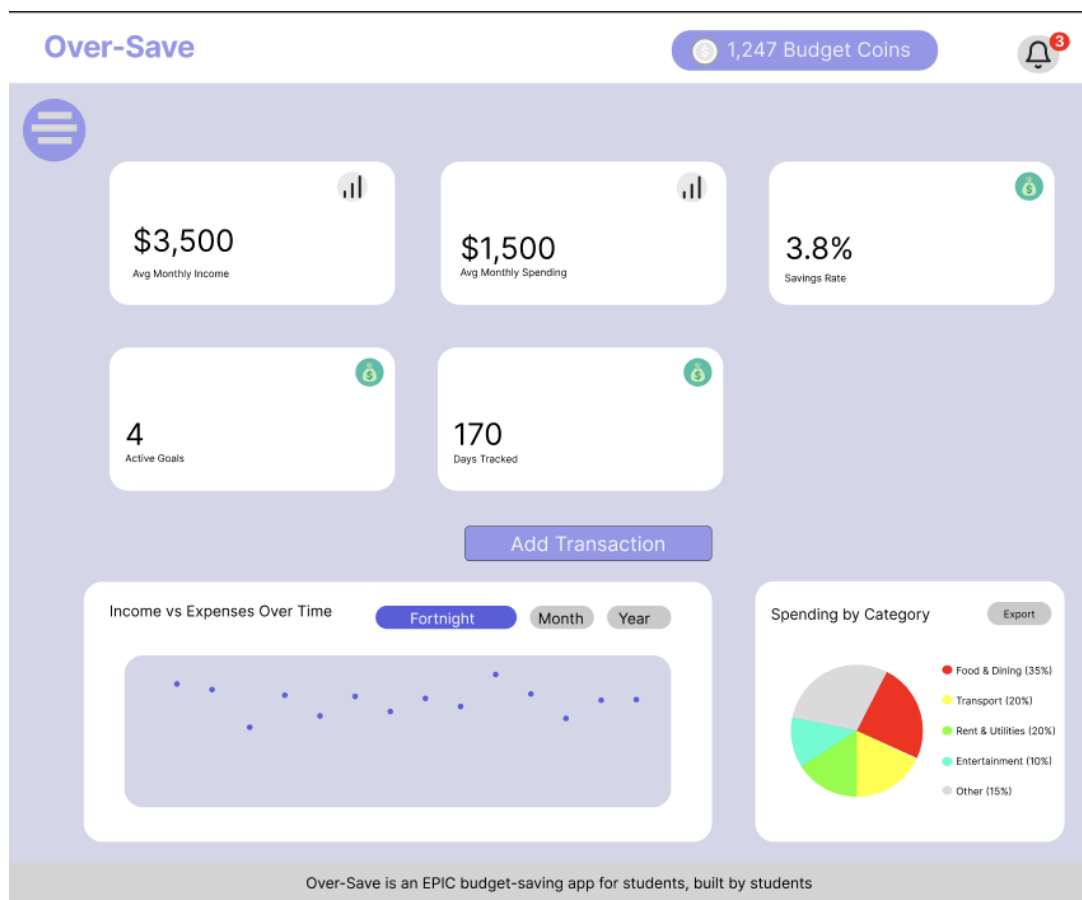


Figure 6: Analytics: Provides detailed spending analysis through visualisations such as bar and pie charts, helping users understand their financial habits.

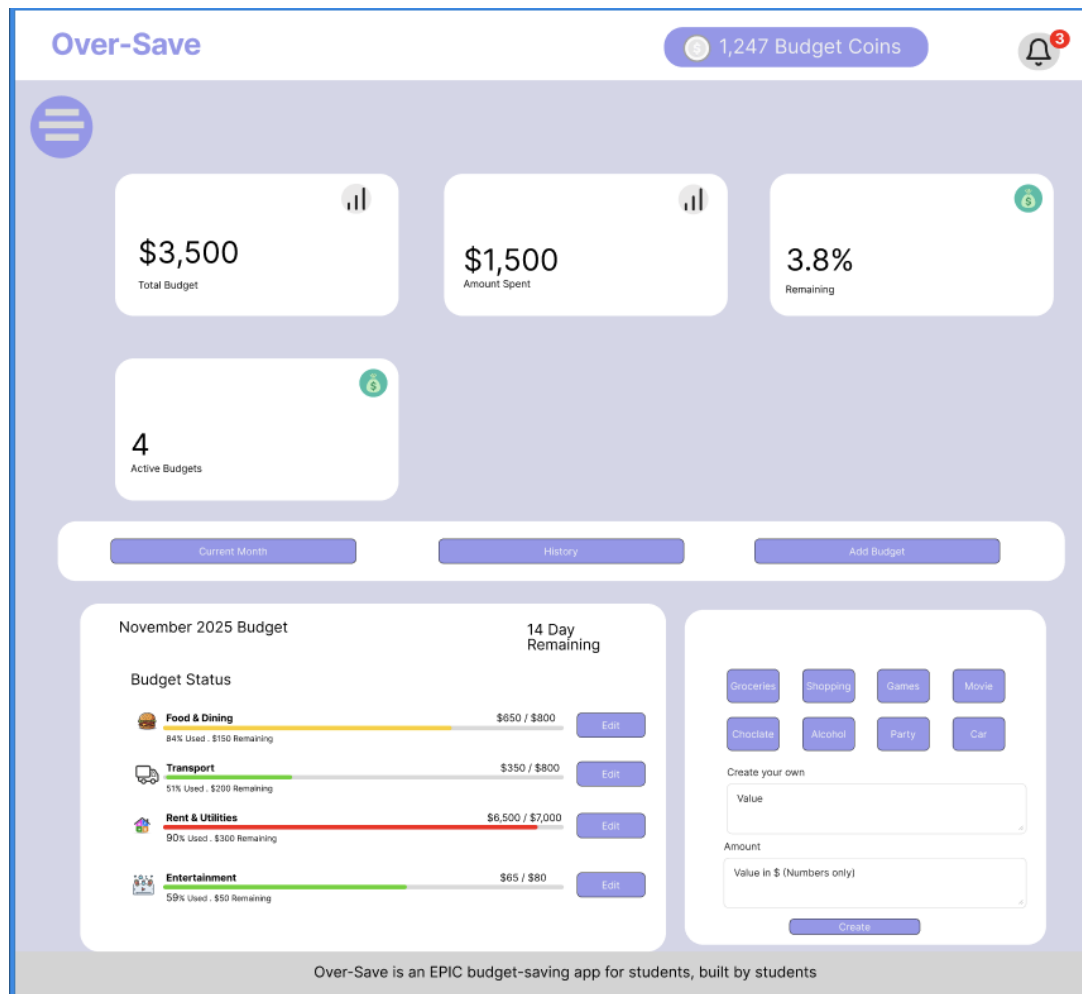


Figure 7: Budget Management: Shows the current month's budget, comparing planned vs actual spending, with progress indicators for each category.

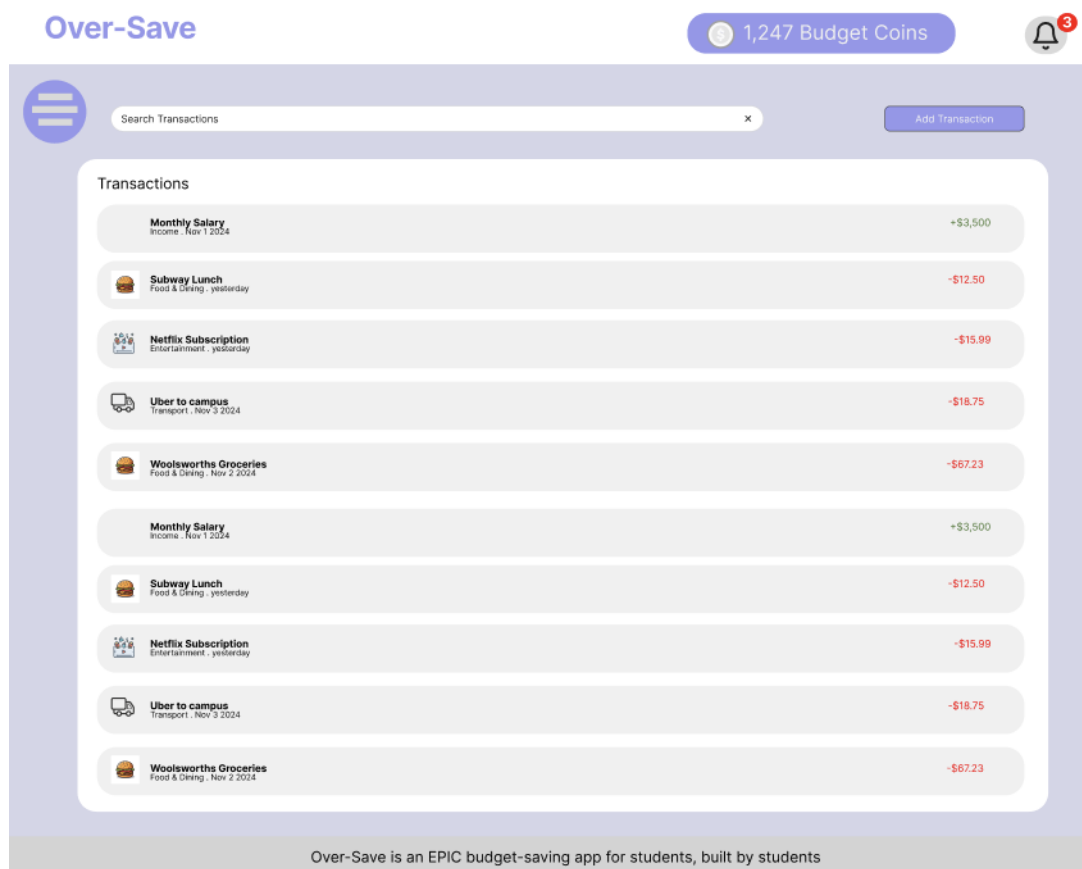


Figure 8: Transactions: Displays the user's transaction history with details on expenses and income.

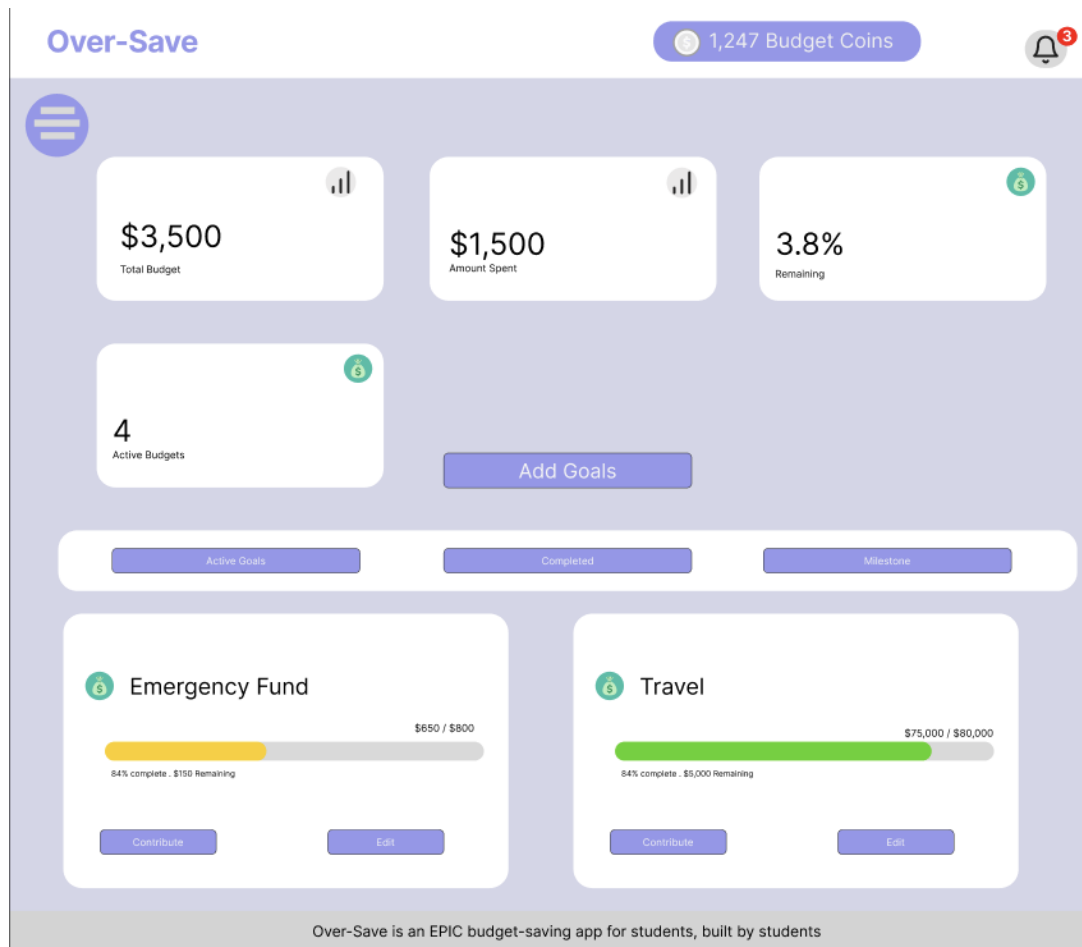


Figure 9: Goals: Allows users to set and track financial goals such as emergency funds or specific savings targets.

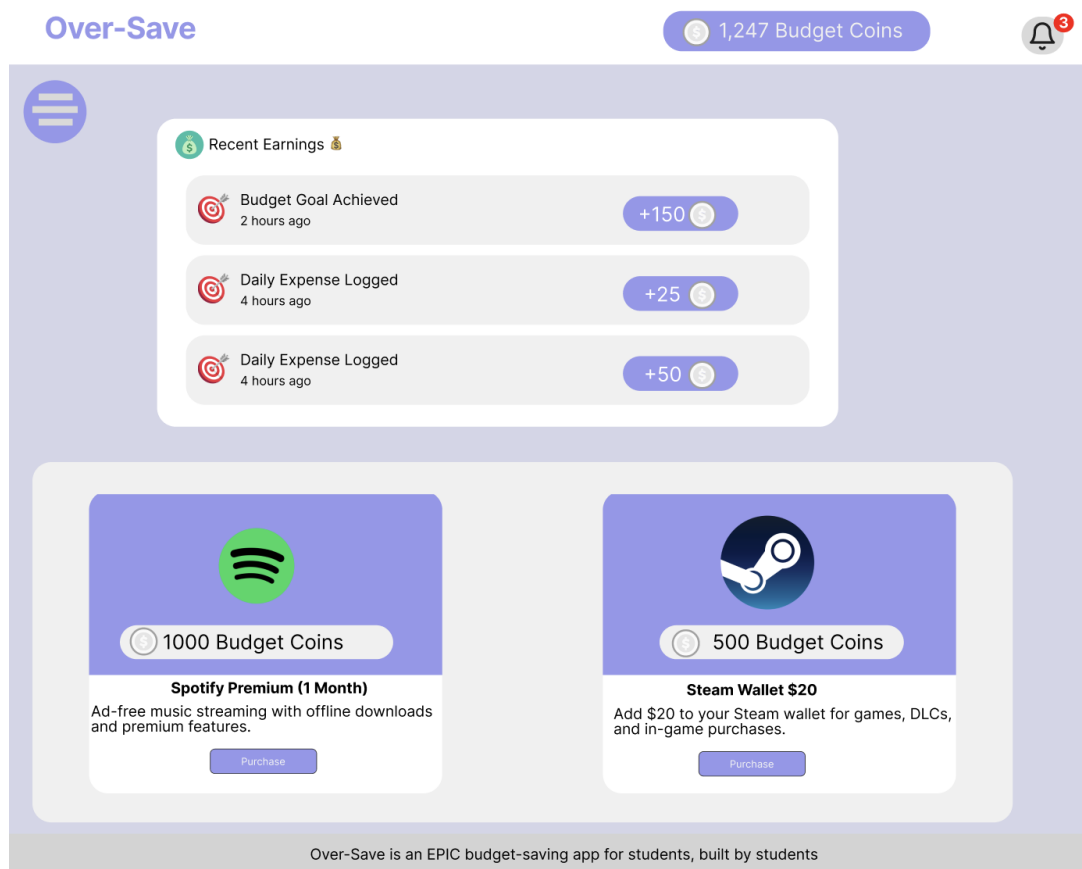


Figure 10: Rewards Shop: A gamified feature where users can redeem points or savings for rewards, enhancing engagement.

5 Project plan

5.1 Project timeline

The project timeline for *OVER-SAVE* extends from early August through late October, providing adequate time for thorough development, testing, and deployment of a production-ready financial management application. The Gantt chart (Figure 11) presents this timeline, tracking 30+ distinct tasks organised into logical development phases with clear dependencies and milestone markers.

5.2 Team roles and responsibilities

Each team member undertakes multiple overlapping roles to ensure collaboration and continuity across the project. The allocation is as follows:

Kartik Nair (510611749)

- Act as Project Manager: oversee project timeline, manage GitHub workflow, review code
- Backend Lead: implement user registration, login, logout, password reset, and session management
- Database schema design and optimization
- Support documentation tasks
- **Responsible FRs:** FR-01 to FR-06

Antriksh Dhand (510415022)

- Frontend Lead: design and implement UI/UX
- Develop dashboard, savings progress, and reports
- Implement notification settings interface
- Conduct frontend validation and usability testing
- **Responsible FRs:** FR-14 to FR-16, FR-20

Taozhao Chen (510019015)

- Backend Developer: implement email verification, password reset, notification preferences
- API Integration: connect external services (OAuth, SMTP, bank statement import)
- Write API documentation and developer guides

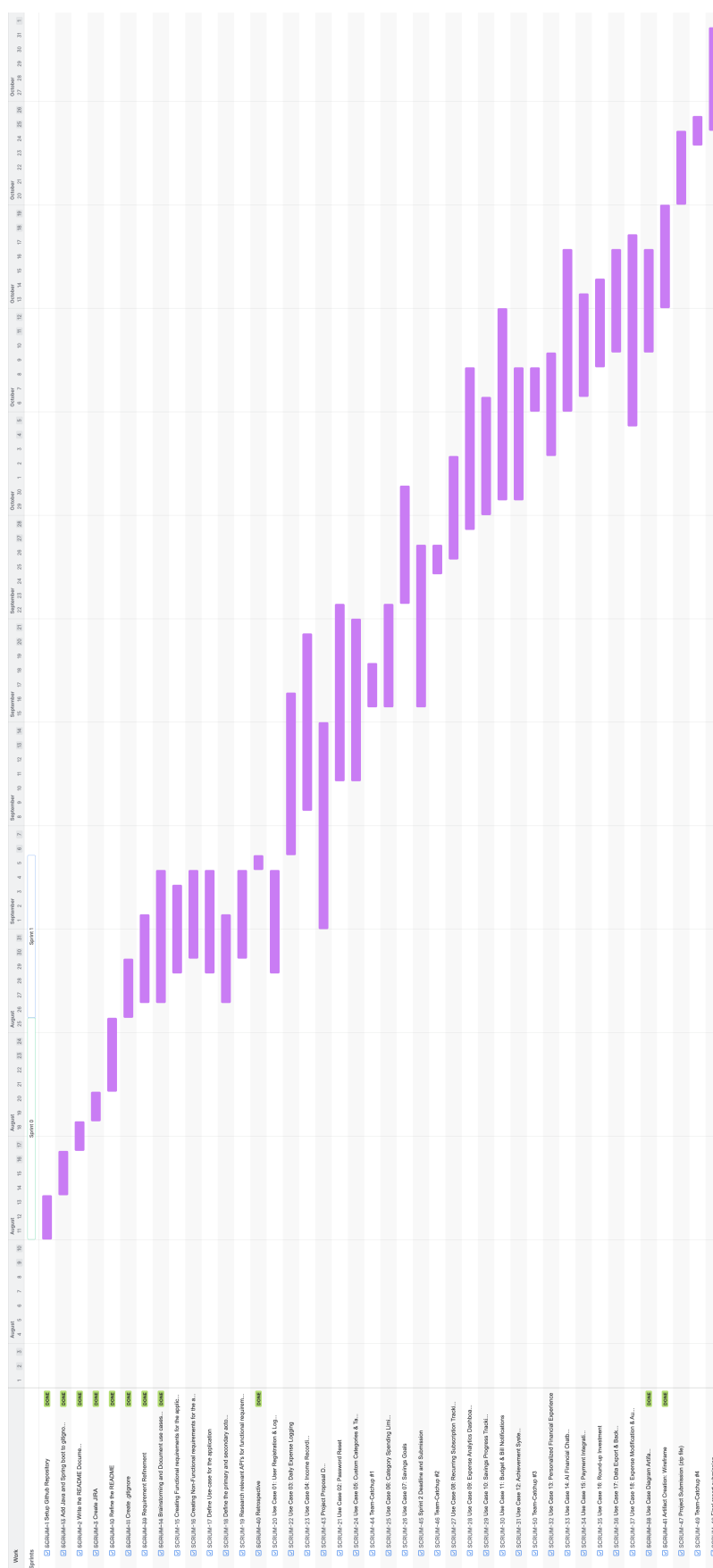


Figure 11: Project plan (Gantt chart) from the beginning of the project.

- Assist with database migration and deployment
- **Responsible FRs:** FR-02, FR-05, FR-18, FR-20

Joey Chen (520076428)

- Database Engineer: optimize database performance and maintain reliability
- Backend Developer: implement expense recording, income recording, category management, recurring subscriptions
- Support frontend components (transaction display, validation)
- **Responsible FRs:** FR-07 to FR-10

Aryan Adhikari (510615596)

- QA Lead: design and execute unit and integration test cases
- Backend Support: implement budget setting, savings goals, and goal contributions
- Documentation Lead: prepare user manuals and project documentation
- Contribute to final presentation and reporting
- **Responsible FRs:** FR-11 to FR-13

References

- Akosah, A., Pantidi, N., & Ferreira, J. (2023, March). Designing mobile applications for personal financial management: An exploratory user-centered design study. In *Fintech research and applications* (pp. 357–383). World Scientific (Europe). https://doi.org/10.1142/9781800612723_0009
- Lund, C., De Silva, M., Plagerson, S., Cooper, S., Chisholm, D., Das, J., Knapp, M., & Patel, V. (2011). Poverty and mental disorders: Breaking the cycle in low-income and middle-income countries. *The Lancet*, 378(9801), 1502–1514. [https://doi.org/10.1016/s0140-6736\(11\)60754-x](https://doi.org/10.1016/s0140-6736(11)60754-x)
- Mission Australia. (2023). Mental health toll: Half of young australians concerned about financial security.
- National Centre for Vocational Education Research (NCVER). (2023). More young people experiencing financial stress.